

Dissecting RLCD: What Reinforcement Learning Does (and Doesn't) Do for Calibrated Typed Decisions

Le Duc Minh

Independent Researcher

Ho Chi Minh City, Vietnam

minh.leduc.0210@gmail.com

Abstract—“Reinforcement Learning for Calibrated Decisions” (RLCD) is the training method behind typed-decision models that return a probability distribution over a closed set of options. The method behind TypeSafe’s Jev is unpublished; the only open reproduction, Laya, adds a noisy-logit policy-gradient term to ordinary soft cross-entropy (CE). This report asks what that RL term actually does. It is a score-function (evolution-strategies) estimate of the gradient of a noise-smoothed proper scoring rule. For the log score, its optimum is provably sharper than the target once the noise is removed for inference, and the gap grows with the noise scale σ . A 13-run ablation that reproduces Laya’s fine-tune on the typed-decisions benchmark shows the effect at full scale: at fixed $\sigma \in \{0.5, 1, 2\}$, the temperature fitted to binary questions rises from 1.2 to 1.7 and the target-referenced calibration error from 0.054 to 0.111. At Laya’s own annealed schedule the RL gradient is 8–87 $\times$ the CE gradient, yet it changes nothing measurable, and CE alone is at least as good on every metric we track. The benchmark’s hard labels turn out to be the teacher’s argmax, which makes hard-label ECE reward over-sharpening; we propose scoring against the target distribution instead.

Index Terms—calibration, proper scoring rules, reinforcement learning, typed decisions, non-autoregressive models, evolution strategies

I. INTRODUCTION

A new class of “System One” models answers structured questions about unstructured state in a single forward pass. No text is generated. Each answer is a probability distribution over a closed, typed set of options: a *choice* among named options, an ordinal *score*, or a binary *noul* probability. TypeSafe’s Jev [1] is the most visible example. Its central promise is calibration: an answer given with 0.8 confidence should be right about 80% of the time. The method credited for it, RLCD [2], has no published paper, reward function, dataset or calibration figure [1], [3].

Laya [4] is an open, transparent attempt at the same interface. It publishes code, checkpoints and an evaluation on the LocalLLaMA/typed-decisions benchmark [5]. Its training loop adds noise to the logits, rewards the noisy distributions with a proper scoring rule, and applies a REINFORCE-style update [6] next to a soft CE term. Laya’s own README reports a raw ECE of 0.466, falling to 0.081 once temperature scaling is applied. On its face, then, the RL objective does not produce calibration by itself.

This report asks what that RL term actually does. There are four findings.

- 1) It is an evolution-strategies estimator [7], [8] of the gradient of a noise-smoothed scoring rule, and as $\sigma \rightarrow 0$ it returns the CE gradient the model already has (Sec. V).
- 2) For the log score and binary questions, the smoothed optimum is sharper than the target at noise-free inference, with a gap that increases strictly in σ ; the same direction holds numerically for Laya’s full reward and $K \leq 77$ options (Sec. V, VI-A).
- 3) A 13-run ablation of the loss reproduces Laya’s fine-tune (Sec. VI-B). The predicted bias appears in full at fixed $\sigma \geq 0.5$, while at Laya’s annealed schedule the RL term dominates the update and still buys nothing over CE-only.
- 4) The benchmark’s gold labels are the teacher’s argmax, so hard-label ECE *improves* as a model over-sharpens; we propose a target-referenced ECE that measures the bias rather than rewarding it.

Nothing in this report claims to know how Jev is built. Statements about Jev come from TypeSafe’s public claims and are labelled as inference.

II. BACKGROUND AND RELATED WORK

Calibration is a statement about a predictor’s own confidence. With $c = \max_i p_i$ as the confidence, the predictor is calibrated if $\Pr(\text{correct} \mid c) = c$. Expected calibration error (ECE) estimates the gap with B equal-width bins [9], [10]:

$$\text{ECE} = \sum_{b=1}^B \frac{|S_b|}{n} \left| \bar{c}_{S_b} - \bar{a}_{S_b} \right|, \quad (1)$$

where $\bar{a}$ is the empirical accuracy in the bin; we use $B = 15$ on $c = \max_i p_i$. Temperature scaling [10] divides the logits by a scalar T fitted by NLL on held-out data. A value $T > 1$ says the raw outputs were over-sharp.

A scoring rule $R(q, t)$ is strictly proper if $\mathbb{E}_{y \sim t} R(q, y)$ is uniquely maximized at $q = t$ [11]. Laya combines three of them: the log score $\sum_i t_i \log q_i$ (its negative is soft CE), the spherical score $\langle t, q \rangle / \|q\|$, and, for ordinal questions, the ranked probability score (RPS) [12]. Brier [13] is $\|q - t\|_2^2$.

TABLE I
PUBLIC CLUES AND DESIGNS CONSISTENT WITH THEM (INFERENCE ONLY).

Public clue	Consistent design
All answers in one query; output tokens “free”	prefill-only single forward pass
≤255 options per choice	closed option set, one logit per option
Negation pairs need not sum to 1	independent per-question readouts
Weak arithmetic, dates, counting	single-pass “System 1” computation
Degrades with unrelated state	attention dilution over a long prefix
Trained only on synthetic data	a controlled data engine with known label distributions

Distilling onto teacher distributions [14] with soft CE therefore already optimizes a strictly proper score.

Three post-training families are relevant here. RLHF optimizes a learned preference reward [15], [16], and post-training of this kind has itself been observed to degrade calibration [17], [18]; RLVR optimizes a verifiable reward [19], often with a group-mean baseline as in GRPO [20]; RLCD, as publicly described, rewards calibrated typed distributions [2]. Perturbing logits and following the score function is Gaussian smoothing, which is to say evolution strategies [7], [8], [21].

III. JEV FROM THE OUTSIDE (INFERENCE)

Table I pairs what TypeSafe has said publicly with the design choices consistent with those claims [1], [3], [22]. Every row is an inference, not a fact. The row that matters here is the last one: if the targets come from a data engine you control, then soft CE already optimizes a proper score exactly, and RL is needed only for rewards that cannot be differentiated, for sequential decisions, or for steering data generation. None of the three applies to Laya’s training loop.

IV. LAYA FROM THE INSIDE

Each question becomes its own sequence: [CLS] <type> question: <instr> [SEP] [MASK] opt₀ [MASK] opt₁ ... [SEP] <state> [SEP]. A ModernBERT-large encoder [23] runs over it, followed by a type embedding, two pre-norm transformer layers and an MLP scorer. The scorer reads one logit z_i from the hidden state at each option’s [MASK], and inference reports $p = \text{softmax}(z/T)$. We rebuilt this model and checked tensor by tensor that Laya’s released weights load into our version (Appendix A).

Algorithm 1 restates the training step from Laya’s typed-decisions fine-tune notebook [4]. Two details in it are easy to miss, and both change the gradient. The spherical weight is 0.75, not the library default of 0.5. And the group-centred advantages are divided by their own standard deviation; using the raw-reward standard deviation instead shrinks the RL gradient by about 90× in our tests.

Here $R(q, t) = \sum_i t_i \log q_i + 0.75 \langle t, q \rangle / \|q\| - \mathbb{K}[\text{score}] \text{RPS}(q, t)$, and sg is a stop-gradient. The noise scale σ is annealed linearly from 0.4 to 0.1 over four epochs. A temperature per question type is fitted afterwards on a

Algorithm 1 Laya’s RLCD training step (one batch).

Require: rows with targets t , noise σ , samples $G=4$, weights

```

 $w_{\text{rl}} = w_{\text{ce}} = 1$ 
1:  $z \leftarrow f_{\theta}(x)$  {masked logits,  $B \times K$ }
2: for  $g = 1$  to  $G$  do
3:    $e_g \sim \mathcal{N}(0, \sigma^2 I)$ ;  $\epsilon_g \leftarrow e_g - \bar{e}_g \mathbf{1}$  {zero mean over valid options}
4:    $q_g \leftarrow \text{softmax}(\text{sg}(z) + \epsilon_g)$ ;  $r_g \leftarrow R(q_g, t)$ 
5: end for
6:  $a_g \leftarrow r_g - \frac{1}{G} \sum_{g'} r_{g'}$ ;  $a \leftarrow a / (\text{std}(a) + 10^{-6})$ 
7:  $L_{\text{RL}} \leftarrow -\frac{1}{GB} \sum_g a_g \log \mathcal{N}(\text{sg}(z) + \epsilon_g \mid z, \sigma^2 I)$ 
8:  $L_{\text{CE}} \leftarrow -\frac{1}{B} \sum t^\top \log \text{softmax}(z)$ 
9: step on  $w_{\text{rl}} L_{\text{RL}} + w_{\text{ce}} L_{\text{CE}}$  (clip 1.0, AdamW)

```

held-out slice; an earlier version fitted it on training items, which issue #186 [24] records.

V. ANALYSIS OF THE RL TERM

A. The RL term estimates a smoothed objective

Let $P = I - \frac{1}{K} \mathbf{1}\mathbf{1}^\top$ and $\epsilon \sim \mathcal{N}(0, \sigma^2 P)$, the zero-mean-projected noise of Algorithm 1. Define

$$J_{\sigma}(z) = \mathbb{E}_{\epsilon} [R(\text{softmax}(z + \epsilon), t)]. \quad (2)$$

The projected noise lies in $\text{range}(P)$, so the pseudoinverse score under $\mathcal{N}(0, \sigma^2 P)$ reduces to ϵ/σ^2 and $\nabla_z \log \mathcal{N}(\text{sg}(z) + \epsilon \mid z, \sigma^2 I) = \epsilon/\sigma^2$, the gradient of L_{RL} is

$$-\nabla_z L_{\text{RL}} = \frac{1}{GB \sigma^2} \sum_g a_g \epsilon_g, \quad (3)$$

a group-baselined Monte-Carlo estimate of the Gaussian-smoothing identity [8], [21]

$$\begin{aligned} \nabla_z J_{\sigma}(z) &= \frac{1}{\sigma^2} \mathbb{E} [R(\text{softmax}(z + \epsilon), t) \epsilon] \\ &= P \mathbb{E} [\nabla_u R(\text{softmax}(u), t) \mid_{u=z+\epsilon}], \end{aligned} \quad (4)$$

where the second equality is Stein’s lemma. For the log score, $\nabla_u R = t - \text{softmax}(u)$, so as $\sigma \rightarrow 0$ the RL term estimates $t - \text{softmax}(z)$, the negative CE gradient itself. In other words, it is a noisy version of a gradient CE already supplies exactly, plus a deviation controlled by σ .

B. The smoothed optimum is over-sharp

For the log score, J_{σ} is concave (an average of concave log-softmax terms), so its maximizers are the stationary points

$$\mathbb{E}_{\epsilon} [\text{softmax}(z_{\sigma}^* + \epsilon)] = t. \quad (5)$$

The *noise-averaged* prediction matches the target, but inference reports the *noise-free* $\text{softmax}(z_{\sigma}^*)$. For binary questions (noul, $K=2$) the consequence can be proved exactly. With projected noise, the logit gap $d = z_1 - z_2$ receives $\eta = \epsilon_1 - \epsilon_2 \sim \mathcal{N}(0, 2\sigma^2)$, and (5) becomes $F_{\sigma}(d^*) := \mathbb{E}[s(d^* + \eta)] = t_1$ with s the logistic function.

Proposition 1. *Let $K = 2$, the log score, and $t_1 \in (\frac{1}{2}, 1)$. For every $\sigma > 0$ there is a unique maximizer d_{σ}^* . It satisfies*

$s(d_\sigma^*) > t_1$, and $s(d_\sigma^*)$ is strictly increasing in σ (symmetrically for $t_1 < \frac{1}{2}$).

Proof. Let $g = s - \frac{1}{2}$. It is odd, increasing, and concave on $[0, \infty)$, with $g''(x) = s(1-s)(1-2s)$ odd and negative for $x > 0$. Fix $d > 0$ and pair $\pm\eta$. If $|\eta| \leq d$, concavity gives $g(d+\eta) + g(d-\eta) \leq 2g(d)$. If $|\eta| > d$, then $g(d+|\eta|) + g(d-|\eta|) = g(|\eta|+d) - g(|\eta|-d)$. This is the increment of a concave function over an interval of length $2d$ starting at $|\eta| - d > 0$, so it is at most $g(2d) - g(0) \leq 2g(d)$, strictly for $\eta \neq 0$. Hence $F_\sigma(d) < s(d)$ for all $d > 0$. The map F_σ is continuous and strictly increasing, with $F_\sigma(0) = \frac{1}{2}$ and $F_\sigma(\infty) = 1$, so $F_\sigma(d) = t_1$ has a unique root $d^* > 0$, and $s(d^*) > F_\sigma(d^*) = t_1$. For monotonicity, write $\eta = \tau Z$ with $\tau = \sqrt{2}\sigma$. By Stein’s lemma, $\partial_\tau \mathbb{E}[g(d+\tau Z)] = \tau \mathbb{E}[g''(d+\tau Z)]$. Because g'' is odd and negative on $(0, \infty)$, and the Gaussian centered at $d > 0$ puts more mass at $+x$ than at $-x$, this expectation is negative. So $F_\sigma(d)$ decreases in σ at fixed d . Keeping $F_\sigma(d_\sigma^*) = t_1$ therefore requires d_σ^* to increase with σ , and s is increasing. $\square$

Remark 1 (size of the bias). A second-order expansion gives $\mathbb{E}[\text{softmax}(z + \epsilon)] = \text{softmax}(z) + \frac{\sigma^2}{2} \sum_{jk} P_{jk} \partial_j \partial_k \text{softmax}(z) + O(\sigma^4)$. The optimum therefore deviates from t by $O(\sigma^2)$. Under Laya’s schedule ($\sigma \leq 0.4$, ending at 0.1) the deviation is 6–100 $\times$ smaller than at $\sigma = 1$. This predicts that Laya’s recipe is nearly unbiased but gains nothing over CE, which is what Sec. VI-B finds.

For $K > 2$ and for Laya’s full reward we rely on numerics (Sec. VI-A). Temperature scaling can undo much of the distortion: at the optimum, noise-free outputs are approximately the target sharpened, and a uniform $T > 1$ reverses most of that.

C. Variance and advantage normalization

After normalization the advantages are $O(1)$ whatever the reward scale, so (3) has norm $\propto \|\epsilon\|/\sigma^2 \propto 1/\sigma$. As σ anneals, the RL term grows relative to CE. It is estimated from only $G=4$ samples, and the batch-wide standard deviation rescales the effective learning rate from step to step. The group mean includes each sample itself, which shrinks the estimate by $(G-1)/G$. None of this changes the smoothed optimum, but it determines how noisy the path to it is (Sec. VI-C).

VI. EXPERIMENTS

All code, configurations and result files are public [25]. Every table and figure is regenerated from JSON result files.

A. E1: Toy smoothing bias

We maximize (2) directly by reparameterized gradients (Adam, 4000 steps, 4096 noise samples per step), which removes estimator noise so that only the smoothed objective’s optimum is measured. For $t = (0.7, 0.2, 0.1)$ and the log score, the noise-free $\max_i \text{softmax}(z^*)_i$ is 0.700, 0.723, 0.780 and 0.905 at $\sigma = 0, 0.5, 1, 2$ (Fig. 1). Laya’s full reward sharpens a little more: 0.923 as a choice question and 0.932 as a score question at $\sigma = 2$. Sweeping $K \in \{2, 5, 20, 77\}$, peaked

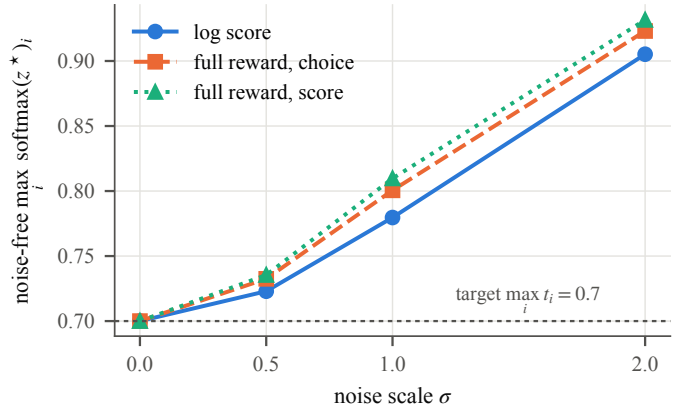


Fig. 1. E1: noise-free confidence at the optimum of the smoothed objective for $t = (0.7, 0.2, 0.1)$. Every reward sharpens beyond the target as σ grows.

and near-uniform Dirichlet targets, and both rewards gives 16 configurations, and in every one of them $\text{KL}(t \| p^*)$ increases monotonically in σ . The effect is largest for peaked mid- K targets and smallest for near-uniform $K=77$.

B. E2: Loss ablation on real data

We replicate Laya’s typed-decisions fine-tune on a single A100. Every run starts from `convaiinnovations/laya` [26], trains for 4 epochs at effective batch 64, and uses AdamW with learning rate $2.5 \cdot 10^{-5}$ on the encoder and 10^{-4} on the head, cosine decay, gradient clipping at 1.0, $G=4$, σ annealed from 0.4 to 0.1, and bf16. The loss is the only thing that changes: CE-only ($w_{\text{rl}}=0$), Laya’s RL+CE, RL-only ($w_{\text{ce}}=0$), and RL+CE at fixed $\sigma \in \{0.5, 1, 2\}$. CE-only, RL+CE and $\sigma=1$ were run with three seeds; the rest with one. All runs share a fixed budget and are evaluated on their final weights, with temperature fitted per (type, option-count bucket) on a held-out 10% of the training cases. The test set holds 400 cases and 2000 questions. Our RL+CE replica reaches accuracy 0.773 ± 0.002 and Brier 0.054 ± 0.001 , against 0.766 and 0.062 in Laya’s README.

One caveat about the benchmark changes how the numbers should be read. The gold label of each question equals $\arg \max_i t_i$ on 98.4% of test rows: it is the teacher’s mode, not an outcome drawn from t . Scored as a predictor, the teacher itself has hard-label ECE 0.326, at mean confidence 0.659 and accuracy 0.984. A model that reproduced t perfectly would look badly calibrated, and over-sharpening would lower its hard-label ECE by pushing confidence toward accuracy. We therefore measure the bias against the target distribution, using a target-referenced ECE that swaps the 0/1 hit of (1) for the target’s probability of the chosen option:

$$\text{ECE}_t = \sum_b \frac{|S_b|}{n} \left| \bar{c}_{S_b} - t[\arg \max p]_{S_b} \right|, \quad (6)$$

which is 0 when $p = t$. Alongside it we report the sharpness gap $\overline{\max p} - \max t$, NLL and Brier against t , the fitted T ,

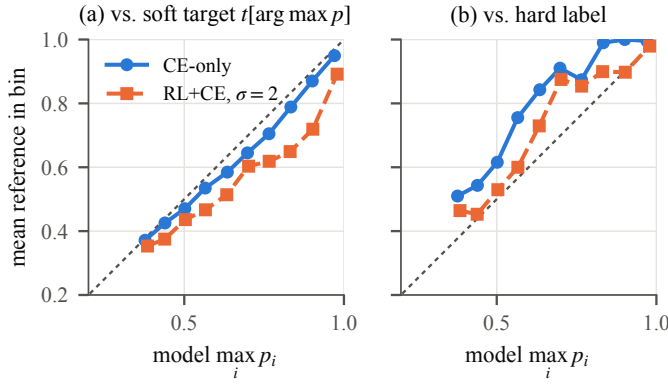


Fig. 2. Reliability curves (bins with ≥ 20 rows) for CE-only and RL+CE at $\sigma=2$. (a) Against the soft target, the $\sigma=2$ model is above the target everywhere (over-sharp). (b) Against hard labels, every model appears under-confident, because the labels are the teacher’s mode.

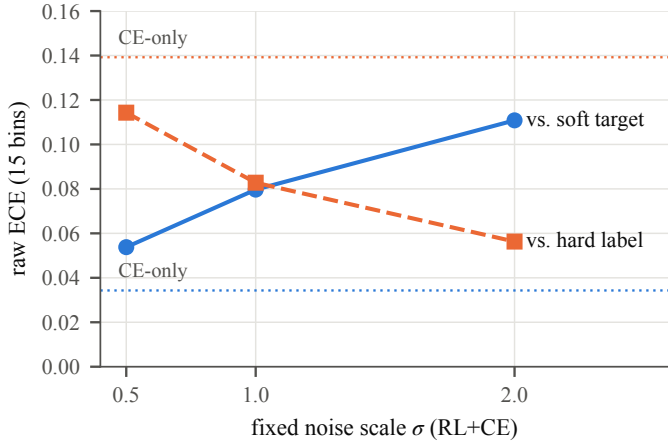


Fig. 3. As fixed σ grows, target-referenced ECE rises while hard-label ECE falls: the same models ranked in opposite order. Dotted lines show CE-only.

and, as the plan requires, raw and post-temperature hard-label ECE. Fig. 2 puts the two references side by side.

At fixed σ the bias appears in full. Table II and Figs. 3–4 show every target-referenced measure getting worse, monotonically. The sharpness gap moves from $+0.001$ to $+0.025$ to $+0.060$, and ECE_t from 0.054 to 0.080 to 0.111 , or $3.3\times$ the CE-only value. Fitted T rises for every type (binary: 1.22 , 1.38 , 1.72), and NLL rises from 0.870 to 0.932 . The three-seed spread at $\sigma=1$ is an order of magnitude smaller than these steps. Over the same runs hard-label ECE falls (0.114 , 0.083 , 0.056), exactly as the caveat predicts. Temperature scaling recovers most of the NLL loss (0.932 to 0.865 at $\sigma=2$), which points to a systematic sharpening rather than a degraded ranking, and accuracy is unchanged within noise.

At Laya’s own schedule the RL term does nothing useful. RL+CE and CE-only are indistinguishable in sharpness (-0.017 vs -0.016), in ECE_t (0.036 vs 0.034) and in fitted T . CE-only wins on NLL for every seed (0.861 vs 0.866 , and still 0.857 vs 0.863 after temperature scaling), on Brier (0.052 vs 0.054) and on accuracy (0.782 vs 0.773). The one-

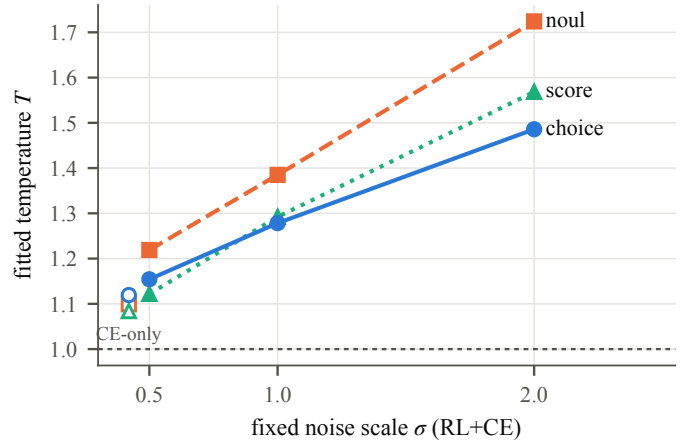


Fig. 4. Fitted temperature per question type versus fixed σ (RL+CE); hollow markers show CE-only. $T > 1$ means the raw outputs are sharper than the targets.

seed RL-only run is the least accurate of all (0.760) but is not over-sharp. This fits the $O(\sigma^2)$ remark: at $\sigma \leq 0.4$ the bias is too small to see, and what is left is estimator noise. The raw hard-label ECE prefers RL+CE (0.132 vs 0.139) for the reason given above.

The sharpening is not uniform. Split by target-entropy tercile (Table III), every model, CE-only included, is under-sharp on peaked targets and over-sharp on flat ones, as if regressing toward typical confidence. Raising σ adds a near-uniform upward shift across the terciles. The near-zero mean gap of CE-only is the sum of two opposite biases, not evidence that no bias exists.

C. E3: Gradient quality

Every 10 optimizer steps we compare (3) against the CE gradient, both with respect to the logits and each drawn from a fresh noise sample (Table IV, Fig. 5). Under Laya’s schedule the RL gradient is $8\times$ the CE gradient at $\sigma=0.4$ and $87\times$ at $\sigma=0.1$, so before clipping it dominates the update; its alignment with CE is only moderate (cosine ≈ 0.65). In the fixed- σ runs, where σ is not entangled with training progress, the ratio scales as $1/\sigma$ (ratio $\times\sigma \approx 5.2, 4.9, 4.3$) and alignment falls from 0.60 to 0.52 to 0.37 as the smoothed gradient drifts from the CE gradient. Size notwithstanding, the RL term leaves the final model about where CE alone would leave it. Everything is consistent with a noisy but roughly unbiased proxy for the CE direction at small σ .

D. Reproducibility

A second run of RL+CE seed 42 matched the first closely (NLL $0.864 / 0.867$, accuracy $0.772 / 0.772$), though not bitwise. An earlier seed-42 run is a different story: it was the first training job on a fresh machine, matched the others exactly at step 1, and then diverged to accuracy 0.697 and NLL 0.936 . We could not pin down the cause; our best guess is the first `torch.compile/Triton` build, since `gcc` had

TABLE II

E2 ON THE TYPED-DECISIONS TEST SET (MEAN $\pm$ STD OVER SEEDS; n = SEEDS). TARGET-REFERENCED COLUMNS FIRST; HARD-LABEL COLUMNS LAST.

Configuration	n	vs. soft target t (raw)				post- T	fitted T			hard-label ECE		Acc.
		$\overline{\max p} - \overline{\max t}$	ECE_t	NLL	Brier		choice	score	noul	raw	post- T	
CE-only ($w_{r1}=0$)	3	-0.016 ± 0.004	0.034 ± 0.002	0.861 ± 0.001	0.052 ± 0.001	0.857 ± 0.001	1.12 ± 0.04	1.08 ± 0.01	1.10 ± 0.03	0.139 ± 0.001	0.158 ± 0.004	0.782 ± 0.004
RL+CE, Laya (σ 0.4 $\rightarrow$ 0.1)	3	-0.017 ± 0.004	0.036 ± 0.002	0.866 ± 0.002	0.054 ± 0.001	0.863 ± 0.002	1.09 ± 0.02	1.06 ± 0.04	1.10 ± 0.01	0.132 ± 0.003	0.146 ± 0.005	0.773 ± 0.002
RL-only ($w_{ce}=0$)	1	-0.024	0.032	0.868	0.055	0.866	1.07	1.06	1.15	0.127	0.141	0.760
RL+CE, $\sigma=0.5$	1	$+0.001$	0.054	0.870	0.057	0.862	1.15	1.12	1.22	0.114	0.144	0.774
RL+CE, $\sigma=1$	3	$+0.025 \pm 0.006$	0.080 ± 0.005	0.890 ± 0.003	0.065 ± 0.001	0.866 ± 0.003	1.28 ± 0.03	1.29 ± 0.02	1.38 ± 0.05	0.083 ± 0.004	0.137 ± 0.003	0.764 ± 0.005
RL+CE, $\sigma=2$	1	$+0.060$	0.111	0.932	0.075	0.865	1.49	1.57	1.72	0.056	0.145	0.774
Teacher t as predictor	–	0	0	–	0	–	–	–	–	0.326	–	0.984

TABLE III

SHARPNESS GAP $\overline{\max p} - \overline{\max t}$ BY TARGET-ENTROPY TERCILE (MEAN $H(t)$: 0.38 / 0.82 / 1.12 NATS).

Configuration	peaked	mid	flat
CE-only ($w_{r1}=0$)	-0.056	-0.017	$+0.026$
RL+CE, Laya (σ 0.4 $\rightarrow$ 0.1)	-0.059	-0.017	$+0.025$
RL-only ($w_{ce}=0$)	-0.067	-0.024	$+0.019$
RL+CE, $\sigma=0.5$	-0.046	$+0.002$	$+0.047$
RL+CE, $\sigma=1$	-0.028	$+0.026$	$+0.076$
RL+CE, $\sigma=2$	$+0.008$	$+0.069$	$+0.104$

TABLE IV

E3: RL VS. CE LOGIT GRADIENTS (n = RUNS LOGGING THE DIAGNOSTIC; THE ANNEALED ROWS ALSO INCLUDE CE-ONLY AND RL-ONLY RUNS, WHOSE RL GRADIENT WAS COMPUTED WITH WEIGHT 0).

σ	n	$\cos(\nabla L_{RL}, \nabla L_{CE})$	$\ \nabla L_{RL}\ /\ \nabla L_{CE}\ $
0.4 (annealed)	8	0.66 ± 0.04	8.4 ± 0.8
0.3 (annealed)	8	0.64 ± 0.04	16.9 ± 3.4
0.2 (annealed)	8	0.66 ± 0.04	34.0 ± 5.3
0.1 (annealed)	8	0.65 ± 0.04	86.6 ± 17.0
0.5 (fixed)	1	0.60	10.4
1.0 (fixed)	3	0.52 ± 0.02	4.9 ± 0.5
2.0 (fixed)	1	0.37	2.2

been installed minutes earlier. We keep it out of the tables but archive it with the results.

VII. TOWARD A PRINCIPLED RLCD PIPELINE

If the goal is calibrated typed distributions, the analysis above suggests the following. Begin with distillation: soft CE against targets from a controlled data engine, with exact labels where the latent state is known, known posteriors where noise is injected, and teacher ensembles otherwise. Save reinforcement learning for what CE cannot do, such as non-differentiable workflow rewards, sequential decisions with temporal-difference targets, and a separate action/escalation head whose utility never reaches the probability head. Then enforce calibration where it can be measured: mine (type, K , length, language) buckets with high target-referenced error and generate more data there; add consistency regularizers for negation pairs (which should sum to one), option permutation and irrelevant context; and fit a temperature per (type, K),

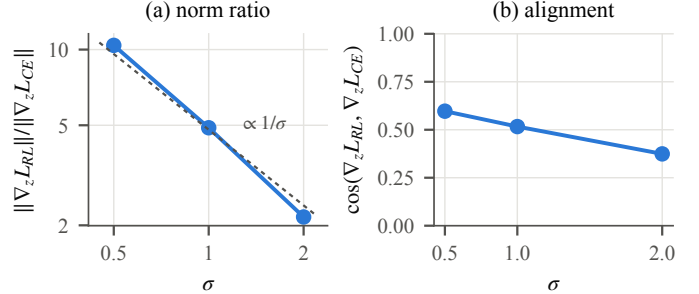


Fig. 5. E3 at fixed σ : (a) the RL/CE gradient-norm ratio follows $1/\sigma$; (b) alignment with the CE gradient falls as σ grows.

treating a large fitted value as a sign that an upstream stage distorted the distribution. Evaluation should always show raw target-referenced calibration beside the post-temperature numbers, and should use labels drawn from distributions, for instance multi-annotator sets such as ChaosNLI [27], whenever calibration on outcomes is claimed. This last part is a proposal; it describes no existing system.

VIII. LIMITATIONS

Jev is closed. Sec. III infers from public claims and tests nothing about Jev itself. The experiments use a single benchmark whose labels are teacher modes, so small accuracy differences are weak evidence; the teacher’s self-agreement ceiling is 0.735, which is why we lead with proper scores and fitted temperatures. Three configurations have a single seed. All runs start from a checkpoint Laya had already trained with RLCD, which can only help RL-only. Proposition 1 covers the log score at $K=2$; the general- K and full-reward cases rest on numerics. The E3 diagnostics are taken at the logits, not the parameters. Model scale is fixed at 421M parameters.

IX. CONCLUSION

Laya’s RL term optimizes a noise-smoothed proper score. As $\sigma \rightarrow 0$ that objective becomes the CE the model already minimizes; for $\sigma > 0$ its optimum is over-sharp at inference. We proved this for binary questions, saw it numerically for Laya’s full reward, and watched it hold at full scale: fitted temperature and target-referenced calibration error both rise monotonically with σ . At Laya’s own small annealed σ , the RL term dominates the gradient and still leaves calibration

unchanged while proper scores get slightly worse than CE-only. For Laya-style typed-decision models, then, the calibration comes from distillation and temperature scaling, not from the RL term. On benchmarks whose labels are teacher modes, hard-label ECE should not be used to argue otherwise.

ACKNOWLEDGMENT

The author thanks the Laya authors for releasing their code, checkpoints, notebook and a candid README, without which this analysis would not have been possible.

REFERENCES

- [1] TypeSafe AI, “Introducing system one models and Jev,” <https://typesafe.ai/blog/introducing-system-one-models-and-jev>, 2026, accessed 2026-09-24.
- [2] System One Models, “RLCD explained,” <https://systemonemodels.org/guides/rlcd-explained/>, 2026, accessed 2026-09-24.
- [3] TechCrunch, “A new kind of AI model from a ChatGPT inventor is thrilling developers,” <https://techcrunch.com/2026/09/18/a-new-kind-of-ai-model-from-a-chatgpt-inventor-is-thrilling-developers/>, 2026, accessed 2026-09-24.
- [4] NandhaKishorM and ConvAI Innovations, “Laya: an open typed-decision model trained with RLCD,” <https://github.com/NandhaKishorM/laya>, 2026, apache-2.0; `laya/common.py` and the typed-decisions fine-tune notebook. Accessed 2026-09-24.
- [5] LocalLLaMA, “typed-decisions,” <https://huggingface.co/datasets/LocalLLaMA/typed-decisions>, 2026, apache-2.0 dataset, Hugging Face. Accessed 2026-09-24.
- [6] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992.
- [7] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever, “Evolution strategies as a scalable alternative to reinforcement learning,” *arXiv preprint arXiv:1703.03864*, 2017.
- [8] Y. Nesterov and V. Spokoiny, “Random gradient-free minimization of convex functions,” *Foundations of Computational Mathematics*, vol. 17, no. 2, pp. 527–566, 2017.
- [9] M. P. Naeni, G. F. Cooper, and M. Hauskrecht, “Obtaining well calibrated probabilities using Bayesian binning,” in *Proc. 29th AAAI Conf. Artificial Intelligence*, 2015, pp. 2901–2907.
- [10] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. 34th Int. Conf. Machine Learning (ICML)*, 2017, pp. 1321–1330.
- [11] T. Gneiting and A. E. Raftery, “Strictly proper scoring rules, prediction, and estimation,” *Journal of the American Statistical Association*, vol. 102, no. 477, pp. 359–378, 2007.
- [12] E. S. Epstein, “A scoring system for probability forecasts of ranked categories,” *Journal of Applied Meteorology*, vol. 8, no. 6, pp. 985–987, 1969.
- [13] G. W. Brier, “Verification of forecasts expressed in terms of probability,” *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.
- [14] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” *arXiv preprint arXiv:1503.02531*, 2015.
- [15] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, “Deep reinforcement learning from human preferences,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [16] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, “Training language models to follow instructions with human feedback,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [17] S. Kadavath, T. Conerly, A. Askell *et al.*, “Language models (mostly) know what they know,” *arXiv preprint arXiv:2207.05221*, 2022.
- [18] OpenAI, “GPT-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [19] N. Lambert, J. Morrison, V. Pyatkin, S. Huang, H. Ivison, F. Brahman *et al.*, “Tulu 3: Pushing frontiers in open language model post-training,” *arXiv preprint arXiv:2411.15124*, 2024.
- [20] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, “DeepSeekMath: Pushing the limits of mathematical reasoning in open language models,” *arXiv preprint arXiv:2402.03300*, 2024.
- [21] S. Mohamed, M. Rosca, M. Figurnov, and A. Mnih, “Monte Carlo gradient estimation in machine learning,” *Journal of Machine Learning Research*, vol. 21, no. 132, pp. 1–62, 2020.
- [22] TypeSafe AI, “TypeSafe documentation: primer, confidence, and jaggedness,” <https://docs.typesafe.ai/>, 2026, accessed 2026-09-24.
- [23] B. Warner, A. Chaffin, B. Clavié, O. Weller, O. Hallström, S. Taghadouini *et al.*, “Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference,” *arXiv preprint arXiv:2412.13663*, 2024.
- [24] Laya contributors, “Issue #186: temperatures fitted on training items,” <https://github.com/NandhaKishorM/laya/issues/186>, 2026, accessed 2026-09-24.
- [25] L. D. Minh, “Dissecting rlcd: code, configs and result files for this report,” <https://github.com/LakoreAI/rlcd-reverse-engineering>, 2026.
- [26] ConvAI Innovations, “Laya model checkpoints,” <https://huggingface.co/convaiinnovations/laya>, 2026, `convaiinnovations/laya`, `laya-typed-decisions`; Hugging Face. Accessed 2026-09-25.
- [27] Y. Nie, X. Zhou, and M. Bansal, “What can we learn from collective human opinions on natural language inference data?” in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 9131–9143.

APPENDIX A REPRODUCTION DETAILS

Laya’s `model.safetensors` holds 206 tensors. Of these, 201 map one-to-one onto our model (encoder, head, type embedding, scorer); the other five are an unused action head and a per-type temperature. Tokenizers are identical to ModernBERT-large’s. Before fine-tuning, the released base checkpoint already scores hard-label ECE 0.215 on a 60-row slice, and its shipped per-type temperatures (1.64 / 1.25 / 1.98) are all > 1 .

Against the notebook we make four deliberate changes: bf16 instead of fp16 with loss scaling; a case-level instead of row-level calibration split; temperature per (type, K -bucket) instead of per type; and one GPU with effective batch 64 instead of two GPUs.

All runs fit in one A100-SXM4-40GB. Each took about 8 minutes (5.5 of them training); the 13 runs plus setup came to about 2.4 GPU-hours.

TABLE V
ALL E2 RUNS (TEST SET; RAW LOGITS).

Run	gap	ECE _t	NLL	Brier	T_c	T_s	T_n	ECE	Acc.
ce_only_seed42	−0.013	0.035	0.860	0.051	1.10	1.09	1.13	0.140	0.785
ce_only_seed43	−0.020	0.032	0.862	0.052	1.09	1.07	1.07	0.139	0.778
ce_only_seed44	−0.014	0.036	0.862	0.051	1.17	1.09	1.09	0.138	0.783
laya_rlce_seed42	−0.016	0.038	0.864	0.053	1.10	1.01	1.11	0.129	0.772
laya_rlce_seed42_rep2	−0.018	0.035	0.867	0.055	1.06	1.08	1.10	0.131	0.772
laya_rlce_seed43	−0.014	0.037	0.867	0.054	1.06	1.09	1.09	0.131	0.775
laya_rlce_seed44	−0.021	0.034	0.867	0.055	1.11	1.06	1.10	0.135	0.772
rl_only_seed42	−0.024	0.032	0.868	0.055	1.07	1.06	1.15	0.127	0.760
rlce_sigma0p5_fixed_seed42	+0.001	0.054	0.870	0.057	1.15	1.12	1.22	0.114	0.774
rlce_sigma1_fixed_seed42	+0.030	0.084	0.888	0.064	1.26	1.28	1.37	0.081	0.770
rlce_sigma1_fixed_seed43	+0.019	0.074	0.893	0.067	1.26	1.29	1.44	0.088	0.761
rlce_sigma1_fixed_seed44	+0.025	0.081	0.889	0.065	1.31	1.31	1.34	0.080	0.762
rlce_sigma2_fixed_seed42	+0.060	0.111	0.932	0.075	1.49	1.57	1.72	0.056	0.774

APPENDIX B THE RLCD LOSS (CONDENSED FROM OUR PORT)

```
k = mask.sum(-1, keepdim=True).clamp(min=2)
eps = torch.randn(G,) + z.shape) * sigma * mask
eps = (eps - eps.sum(-1, keepdim=True) / k) * mask
zs = z.detach().unsqueeze(0) + eps
q = torch.softmax(zs.masked_fill(~mask, -1e4), -1)
with torch.no_grad():
    r = proper_reward(q, t, qtype, mask, w_sph=0.75)
    adv = r - r.mean(0, keepdim=True)
    adv = adv / (adv.std() + 1e-6)
logp = -(((zs - z) ** 2) * mask).sum(-1) / (2 * sigma**2)
loss_rl = -(adv * logp).mean()
loss_ce = -(t * torch.log_softmax(z, -1)).sum(-1).mean()
loss = w_rl * loss_rl + w_ce * loss_ce
```